

1. Hello.c

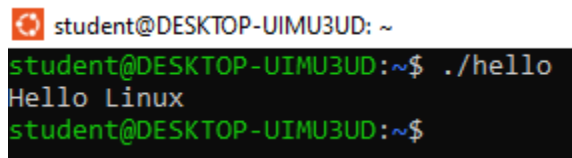
```
#include<stdio.h>
```

```
int main()
{
    printf("Hello Linux\n");
    return 0;
}
```

Compile: gcc hello.c -o hello

./hello

OUTPUT

A terminal window with a black background and green text. The prompt is 'student@DESKTOP-UIMU3UD: ~'. The user enters './hello' and the output is 'Hello Linux'. The prompt returns to 'student@DESKTOP-UIMU3UD: ~\$'.

```
student@DESKTOP-UIMU3UD: ~
student@DESKTOP-UIMU3UD:~$ ./hello
Hello Linux
student@DESKTOP-UIMU3UD:~$
```

2. Read User Name

```
#include<stdio.h>
```

```
int main()
{
    char name[30];

    printf("Enter your name: ");
    scanf("%s",name);

    printf("Welcome %s\n",name);

    return 0;
}
```

OUTPUT

```
student@DESKTOP-UIMU3UD:~$ nano name.c
student@DESKTOP-UIMU3UD:~$ gcc name.c -o name
student@DESKTOP-UIMU3UD:~$ ./name
Enter your name: Abhishek
Welcome Abhishek
student@DESKTOP-UIMU3UD:~$
```

3. Display Process ID

```
#include<stdio.h>
#include<unistd.h>

int main()
{
    printf("Process ID = %d\n",getpid());

    return 0;
}
```

OUTPUT

```
student@DESKTOP-UIMU3UD:~$ nano pid.c
student@DESKTOP-UIMU3UD:~$ gcc pid.c -o pid
student@DESKTOP-UIMU3UD:~$ ./pid
Process ID = 45
student@DESKTOP-UIMU3UD:~$
```

4. Fork()

```
#include<stdio.h>
#include<unistd.h>

int main()
{
    fork();

    printf("Hello\n");

    return 0;
}
```

OUTPUT

```
student@DESKTOP-UIMU3UD:~$ nano fork_test.c
student@DESKTOP-UIMU3UD:~$ gcc fork_test.c -o fork_test
student@DESKTOP-UIMU3UD:~$ ./fork_test
Hello
Hello
student@DESKTOP-UIMU3UD:~$ _
```

5. Parent and Child

```
#include<stdio.h>
#include<unistd.h>

int main()
{
    pid_t pid;

    pid=fork();

    if(pid==0)
    {
        printf("I am Child Process\n");
    }
    else
    {
        printf("I am Parent Process\n");
    }

    return 0;
}
```

OUTPUT

```
student@DESKTOP-UIMU3UD:~$ nano parent_child.c
student@DESKTOP-UIMU3UD:~$ gcc parent_child.c -o parent_child
student@DESKTOP-UIMU3UD:~$ ./parent_child
I am Parent Process
I am Child Process
student@DESKTOP-UIMU3UD:~$
```

6. Display parent and Child PID

```
#include<stdio.h>
#include<unistd.h>

int main()
{
    pid_t pid;

    pid=fork();

    if(pid==0)
    {
        printf("Child PID = %d\n",getpid());
        printf("Parent PID = %d\n",getppid());
    }
    else
    {
        printf("Parent PID = %d\n",getpid());
    }

    return 0;
}
```

OUTPUT

```
student@DESKTOP-UIMU3UD:~$ nano parent_child_pid.c
student@DESKTOP-UIMU3UD:~$ gcc parent_child_pid.c -o parent_child_pid
student@DESKTOP-UIMU3UD:~$ ./parent_child_pid
Parent PID = 97
Child PID = 98
Parent PID = 1
student@DESKTOP-UIMU3UD:~$ _
```

7. Wait()

```
#include<stdio.h>
#include<unistd.h>
#include<sys/wait.h>

int main()
{
    pid_t pid;

    pid=fork();

    if(pid==0)
    {
        printf("Child Running\n");
    }
    else
    {
        wait(NULL);

        printf("Parent Running After Child\n");
    }

    return 0;
}
```

OUTPUT

```
student@DESKTOP-UIMU3UD:~$ nano wait_example.c
student@DESKTOP-UIMU3UD:~$ gcc wait_example.c -o wait_example
student@DESKTOP-UIMU3UD:~$ ./wait_example
Child Running
Parent Running After Child
student@DESKTOP-UIMU3UD:~$
```

8. Wait two()

```
#include<stdio.h>
#include<unistd.h>

int main()
{
    printf("Before exec()\n");

    execlp("ls","ls",NULL);

    printf("After exec()\n");

    return 0;
}
```

OUTPUT

```
student@DESKTOP-UIMU3UD:~$ nano exec_example.c
student@DESKTOP-UIMU3UD:~$ gcc exec_example.c -o exec_example
student@DESKTOP-UIMU3UD:~$ ./exec_example
Before exec()
Hello.c      exec_example  fork_exec  fork_test  hello  name.c      parent_child.c  parent_child_pid.c  pid.c
wait_example.c  zombie.c
Hello.c.save  exec_example.c  fork_exec.c  fork_test.c  name  parent_child  parent_child_pid  pid                wait_example  zombie
student@DESKTOP-UIMU3UD:~$
```

9. fork() + exec()

```
#include<stdio.h>
#include<unistd.h>
#include<sys/wait.h>

int main()
{
    pid_t pid;

    pid=fork();

    if(pid==0)
    {
        execlp("date","date",NULL);
    }
    else
    {
        wait(NULL);

        printf("Child Completed\n");
    }

    return 0;
}
```

OUTPUT

```
student@DESKTOP-UIMU3UD:~$ nano fork_exec2.c
student@DESKTOP-UIMU3UD:~$ gcc fork_exec2.c -o fork_exec2
student@DESKTOP-UIMU3UD:~$ ./fork_exec2
Sun Aug  9 06:10:10 UTC 2026
Child Completed
student@DESKTOP-UIMU3UD:~$
```